EpaDash — Task 1

Merchant Discovery and Product Catalogue APIs

Implementation starter and integration guide | Version 0.1

1. Scope

Public read-only APIs for merchant listing/search/category filtering/profile, merchant product listing, global product search/filtering and product detail.

2. Endpoints

Method	Endpoint	Purpose
GET	/api/v1/merchants	List/search active public merchants
GET	/api/v1/merchants/{slug}	Public merchant profile
GET	/api/v1/merchants/{slug}/products	Published products for a merchant
GET	/api/v1/products	Global public catalogue
GET	/api/v1/products/{productId}	Published product detail

3. Visibility rules

Merchants are returned only when status = ACTIVE and isPublic = true. Products are returned only when status = PUBLISHED, isAvailable = true, and the parent merchant is ACTIVE and public. These predicates are applied in database queries.

Public responses must exclude owner IDs, KYC evidence, authentication fields, supplier cost, internal notes, payment credentials, audit metadata and private delivery-zone geometry.

4. Search, filters and pagination

Merchant filters: q, categoryId, page, limit, sort. Product filters add merchantId, minPrice, maxPrice and available. Page defaults to 1; limit defaults to 20 and is capped at 100. Search is case-insensitive for configured fields.

5. Operating status and delivery availability

The proposed database does not define merchant timezone or holiday/special-hours rules. The starter returns weekly hours but does not claim isOpenNow. Delivery availability is destination-specific; active zone labels may be listed, but a boolean requires customer destination input and approved spatial rules.

6. Inventory availability

When inventory exists, inStock is derived from quantityOnHand minus quantityHeld. Missing inventory returns null until product policy defines whether it means unlimited or unknown stock. Stock reservation must be transactional.

7. Integration steps

1. Copy discovery module and visibility helper into the Task 2 backend.

2. Reconcile Prisma names using docs/integration-checklist.md.

3. Mount createDiscoveryRouter(prisma) at /api/v1 in the existing Express app.

4. Merge unit tests and add PostgreSQL-backed repository/API tests.

5. Run typecheck, lint, migration checks and tests before the pull request.

8. Completion status

Code starter, API contract, OpenAPI file, integration checklist, acceptance/test plan and pure policy tests are prepared. No installation, compilation, mounting, database connection or test execution has been performed. This is not yet a fully

implemented, tested or production-ready feature.

9. Package contents

src/modules/discovery; src/lib/discovery-visibility.ts; tests/discovery-visibility.test.ts; openapi.yaml; docs/api-contracts.md; docs/integration-checklist.md; docs/acceptance-and-test-plan.md; project-progress-update.md.